

Deep Learning에서 LLM까지

뉴런의 단순한 수식이 언어 모델로 확장되는 과정

$H(x) = wx + b$ 에서 시작하는 인공지능의 학습 원리

오늘 연결할 하나의 흐름

1. 뉴런과 퍼셉트론 — 신호를 합해 출력을 결정한다
2. DNN과 RNN — 층과 상태값으로 더 복잡한 문제를 다룬다
3. NLP와 임베딩 — 언어를 숫자로 바꾼다
4. 어텐션과 트랜스포머 — 문맥 관계를 직접 학습한다
5. BERT와 GPT — 이해와 생성을 대규모로 확장한다

핵심 질문: 모델이 커지고 복잡해져도 변하지 않는 학습 원리는 무엇일까?

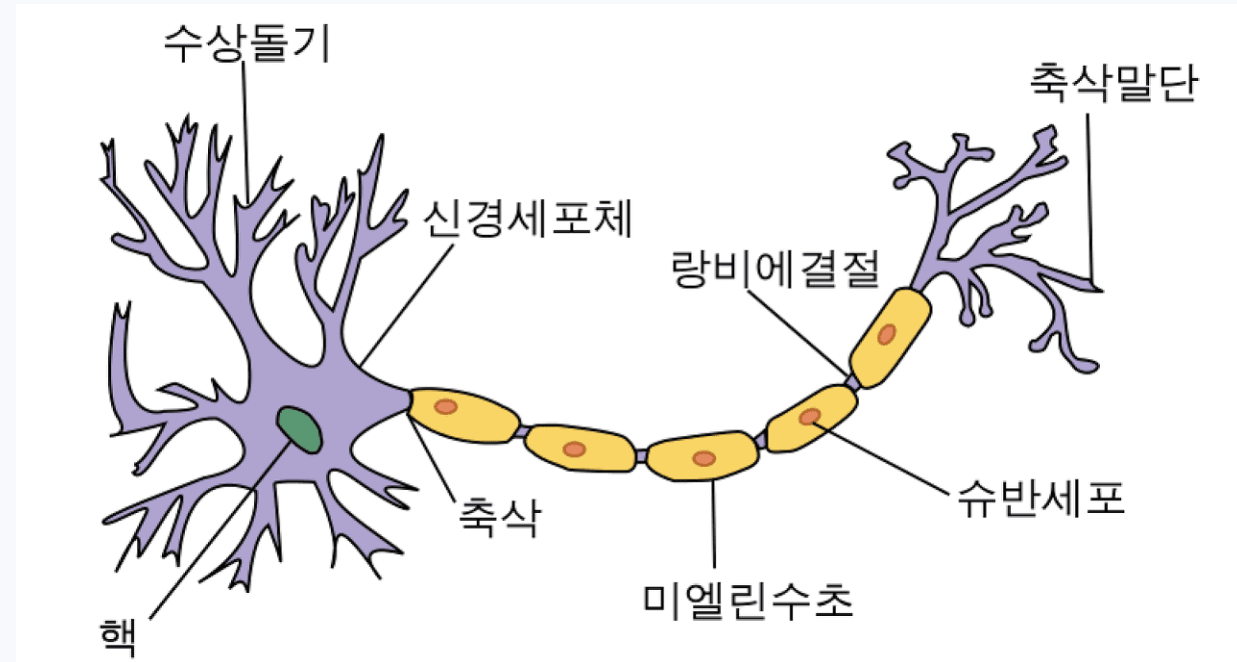
1. 뉴런에서 퍼셉트론으로

생물학적 신호 전달을 계산 가능한 모듈로 바꾸다

뉴런은 신호를 받고, 판단하고, 전달한다

- **수상돌기**: 여러 뉴런으로부터 신호를 받는다
- **신경세포체**: 들어온 정보를 종합한다
- **축삭**: 처리된 신호를 다음 뉴런으로 전달한다

여러 자극의 합이 기준을 넘으면 전달되고, 그렇지 않으면 소멸한다.

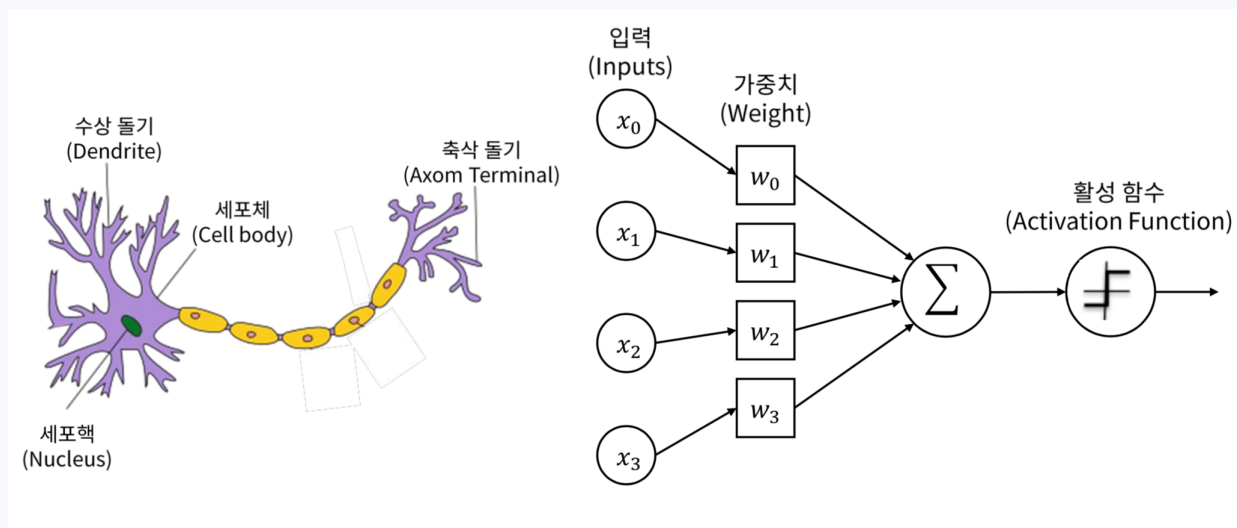


퍼셉트론은 뉴런을 수식으로 모방한다

기본 가설식

$$H(x) = w_1x_1 + w_2x_2 + \dots + b$$

- x : 입력 신호
- w : 각 신호의 중요도를 나타내는 가중치
- b : 판단 기준을 조정하는 편향
- 활성화 함수: 합산 결과를 출력으로 바꾼다



첫 검증: AND와 OR를 학습할 수 있는가?

- **AND**: 두 입력이 모두 1일 때만 1
- **OR**: 하나라도 1이면 1
- 목표는 진리표를 만족하는 w_1 , w_2 , b 를 찾는 것

입력과 정답을 반복해서 보여주며 오차가 줄어드는 방향으로 가중치를 조정한다.

AND



Inputs		Output
A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

OR



Inputs		Output
A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

틀릴 때마다 더 나은 가중치로 이동한다

1. 현재 가중치로 출력을 계산한다
2. 정답과의 오차를 구한다
3. 오차가 줄어드는 방향을 찾는다
4. 가중치를 조금씩 갱신한다

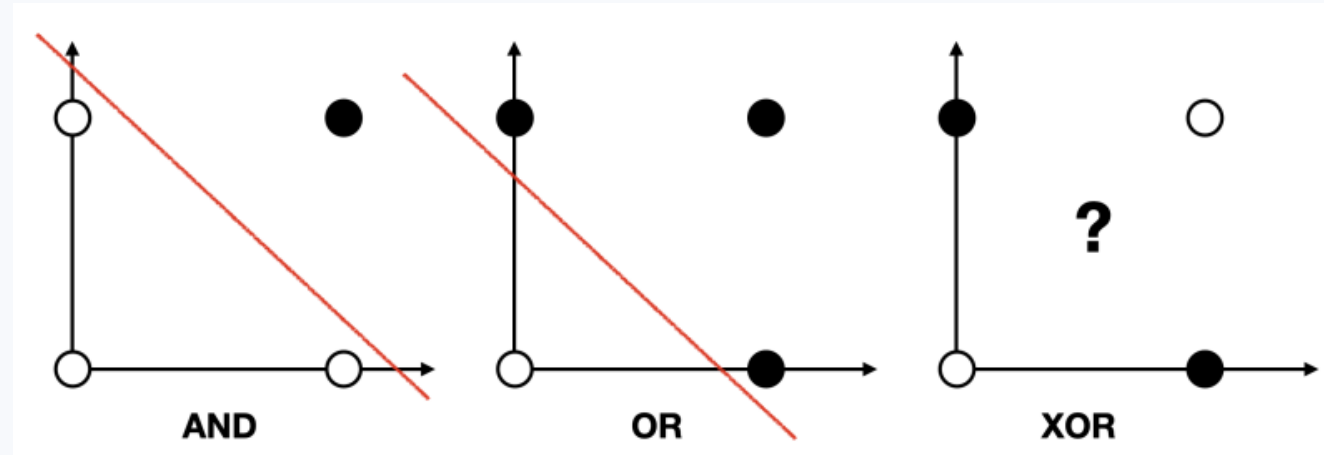
경사하강법은 “어느 방향으로 고칠지” 알려준다



XOR는 하나의 직선으로 분리할 수 없다

- AND와 OR는 직선 하나로 두 부류를 나눌 수 있다
- XOR는 출력이 1인 점들이 대각선으로 떨어져 있다
- 단일 퍼셉트론은 **선형 경계 하나**만 만들 수 있다

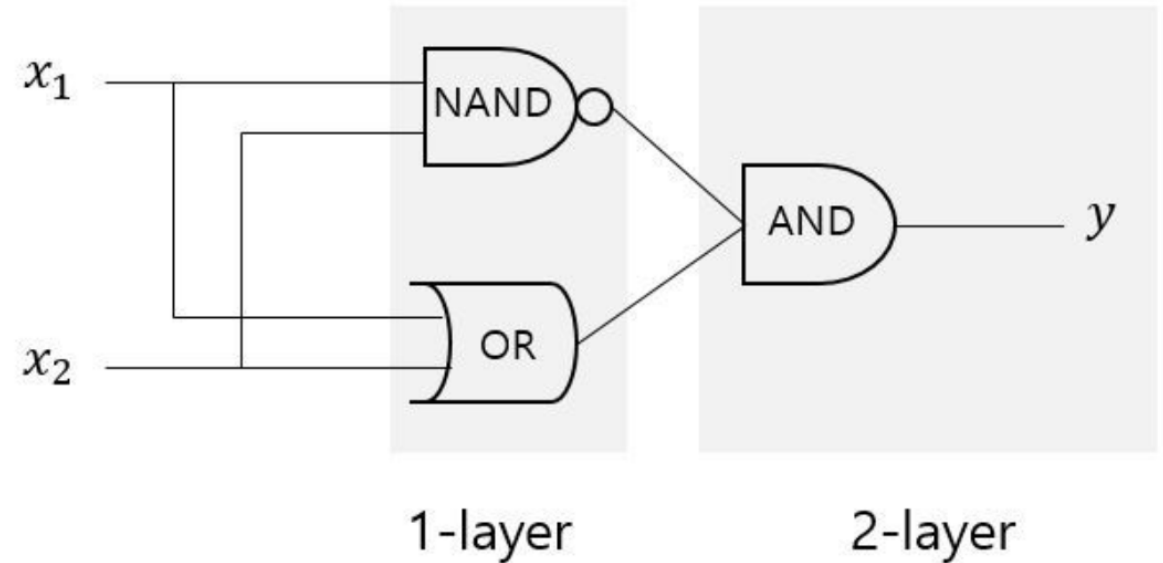
아무리 반복 학습해도 구조가 표현할 수 없는 답은 찾지 못한다.



여러 퍼셉트론을 층으로 연결 하면 XOR를 풀 수 있다

1. 첫 번째 층이 입력을 서로 다른 방식으로 나눈다
2. 활성화 함수가 중간 출력을 만든다
3. 두 번째 층이 중간 출력을 다시 결합한다
4. 최종 출력이 XOR 진리표를 만족한다

단일 직선의 한계를 여러 경계의 조합으로 넘는다



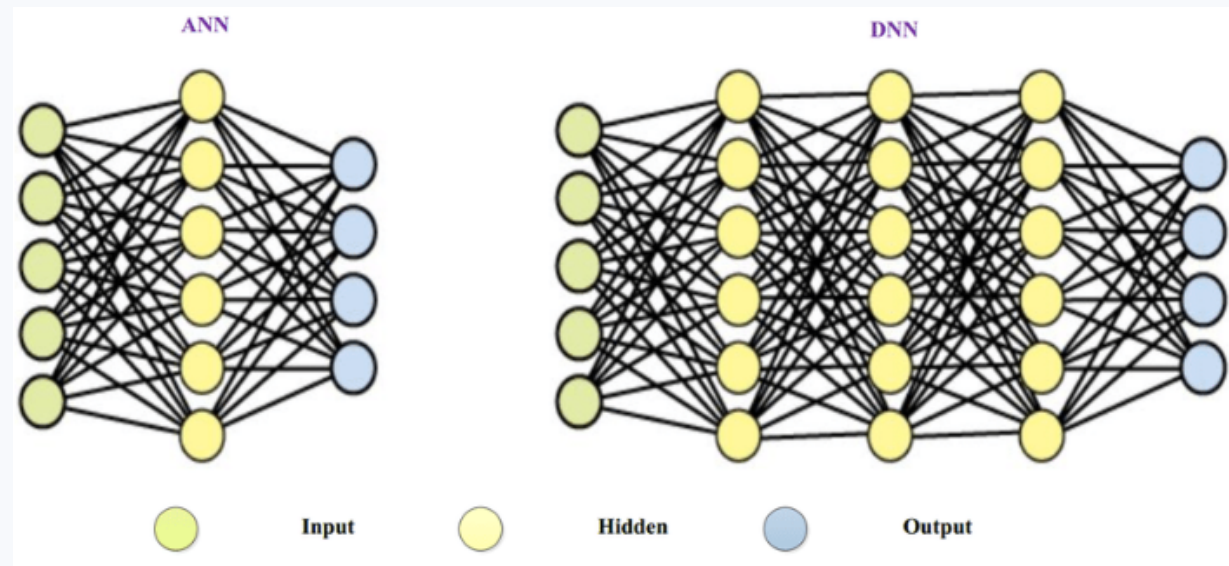
넓어지고 깊어지면 무엇이 달라질까?

퍼셉트론의 수와 층이 늘어나면 모델이 표현할 수 있는 가설의 범위가 커진다.

Wide & Deep: 더 많은 가설을 표현한다

- **Wide**: 한 층의 뉴런이 많아져 더 많은 특징을 다룬다
- **Deep**: 층이 많아져 특징을 단계적으로 변환한다
- **Hidden layer**: 입력을 문제 해결에 유용한 표현으로 바꾼다

다층 신경망은 단순한 경계를 넘어 복잡한 패턴을 학습한다.



DNN은 레고 블록처럼 표현력을 확장한다

- 블록이 많을수록 만들 수 있는 구조가 다양해진다
- 뉴런과 층이 많을수록 학습할 가중치도 많아진다
- 더 다양한 가설 중에서 문제에 적합한 해를 찾을 수 있다

깊이는 가능성을 키우지만, 자동으로 정답을 보장하지는 않는다



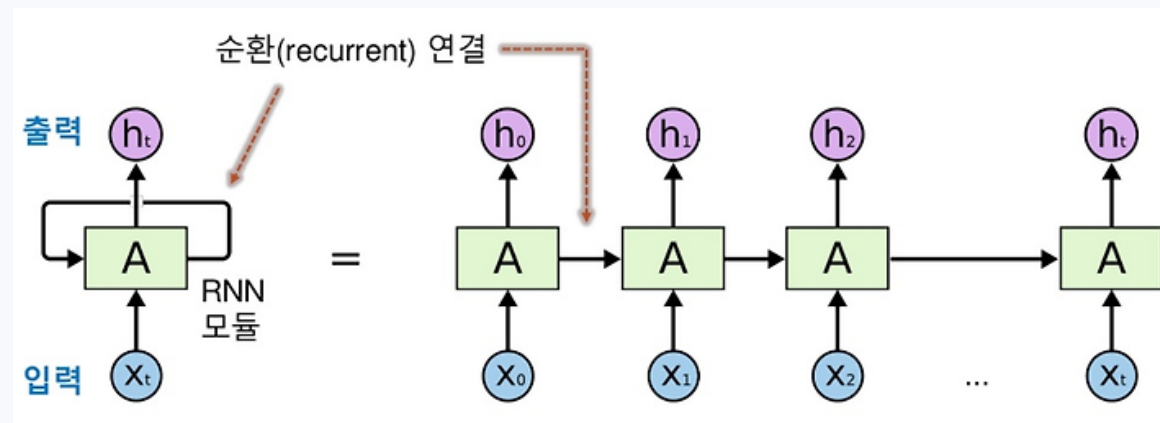
2. DNN에서 RNN으로

독립적인 입력을 넘어 순서와 맥락을 다루다

RNN은 이전 정보를 다음 시점으로 전달한다

- 현재 입력 x_t 와 이전 상태 h_{t-1} 를 함께 사용한다
- 새 상태 h_t 에는 지금까지의 정보가 누적된다
- 같은 모듈을 시간 순서에 따라 반복 적용한다

$$h_t = f(x_t, h_{t-1})$$



RNN이 잘 맞는 데이터에는 순서가 있다

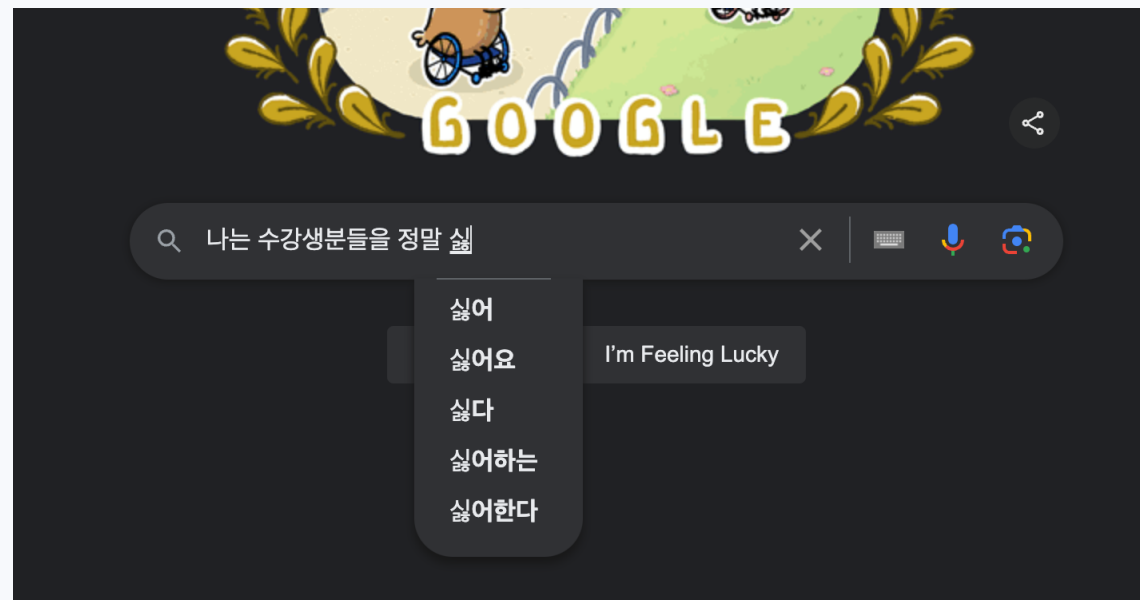
데이터	앞뒤 관계의 예
문장	앞 단어가 다음 단어의 의미에 영향을 준다
음성	앞 소리와 뒤 소리가 이어져 단어가 된다
주가·기온	이전 시점의 값이 현재와 연결된다
음악	앞의 리듬과 멜로디가 다음 구절을 만든다

“동해물과 백두산이 …” 다음에 무엇이 올지 예측하려면 앞의 맥락이 필요하다.

RNN은 초기 언어 서비스의 기반이 되었다

- 자동 완성과 예측 입력
- 음성 인식
- 기계 번역
- 챗봇
- 작곡·시·대본 생성

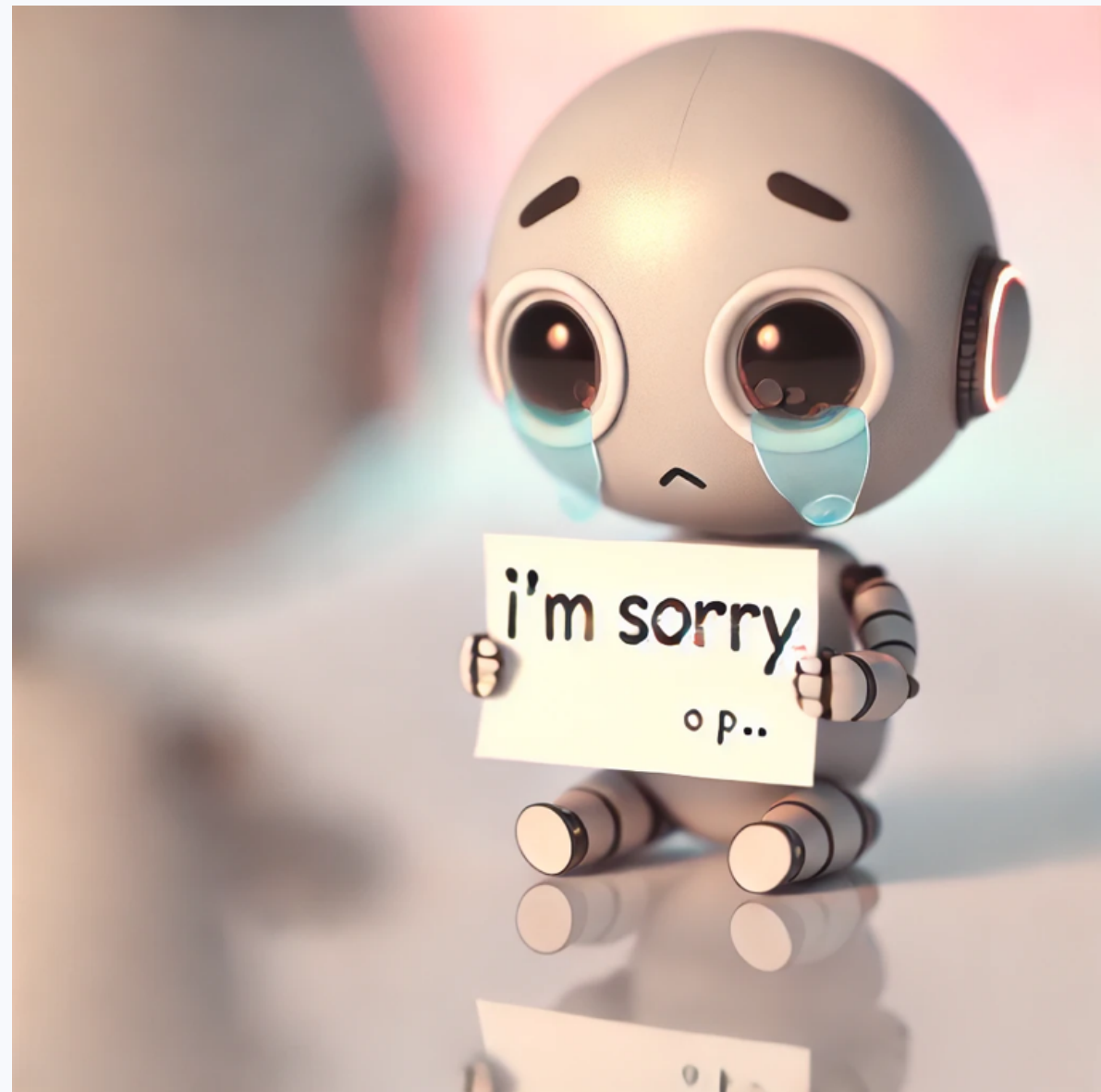
상태값이 문맥을 이어 주기 때문에 가능한 기능들이다



하지만 문장이 길어질수록 앞의 정보가 희미해진다

- 상태값은 새 입력을 받을 때마다 계속 갱신된다
- 오래된 정보는 여러 단계를 거치며 영향력이 약해질 수 있다
- 긴 문장의 처음과 끝을 연결하기 어려워진다

하나의 상태값에 문장 전체를 압축하는 방식이 병목이 된다.



3. 언어를 숫자로 바꾸는 방법

NLP, 토큰, 임베딩 그리고 전이학습

NLP의 첫 단계는 언어를 계산 가능한 단위로 바꾸는 것이다

1. 문장을 작은 단위인 **토큰**으로 나눈다
2. 토큰마다 고유한 번호를 부여한다
3. 토큰 집합을 **어휘 목록**으로 관리한다
4. 숫자 표현을 모델의 입력으로 사용한다

문자 그대로가 아니라 숫자로 바뀐 언어를 계산한다



자연어 처리는 언어를 다루는 여러 기능을 만든다

기능	모델이 해야 하는 일
문서 분류	문서가 어느 범주에 속하는지 판단
기계 번역	원문의 맥락을 유지하며 다른 언어로 변환
질의응답	질문의 요점을 파악하고 답을 구성
문장 생성	문맥에 맞는 다음 단어를 이어서 생성

다음 단어를 예측하는 모델이 다양한 언어 생성 기능의 기반이 된다.

딥러닝은 NLP의 성능을 크게 끌어올렸다

- **기계 번역**: 단어 대 단어 치환에서 문맥 중심 번역으로
- **음성 인식**: 연속적인 음성을 더 정확하게 텍스트로
- **OCR**: 이미지 속 문자를 높은 정확도로 추출
- **생성형 AI**: 자연스러운 문장 생성과 대화로 확장

복잡한 언어 패턴을 데이터에서 직접 학습하기 시작했다

워드 임베딩은 단어의 특성을 벡터에 담는다

- 단순한 번호는 단어 사이의 관계를 표현하지 못한다
- 임베딩은 의미와 문법적 특성을 여러 숫자로 표현한다
- 비슷한 단어는 벡터 공간에서 가까워지도록 학습한다

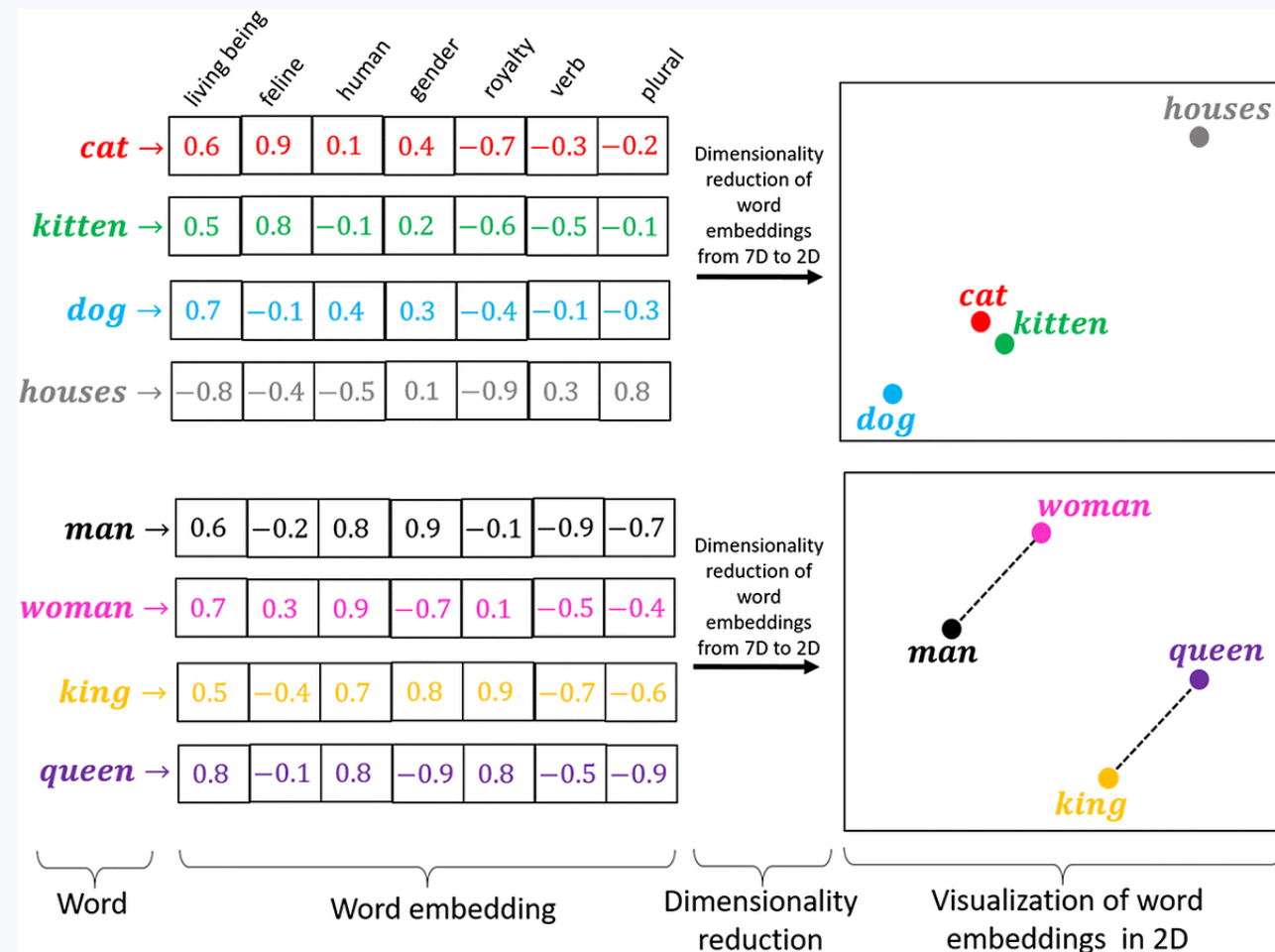
어휘 목록을 번호표가 아니라 “계산 가능한 사전”으로 바꾸는 과정



차원은 단어를 설명하는 특성의 수다

- 3차원 임베딩: 단어 하나를 세 개의 수로 표현
- 고차원 임베딩: 더 많은 특성 축으로 세밀하게 표현
- 각 축을 사람이 직접 해석하기는 어렵다
- 중요한 것은 벡터 사이의 **거리와 방향**이다

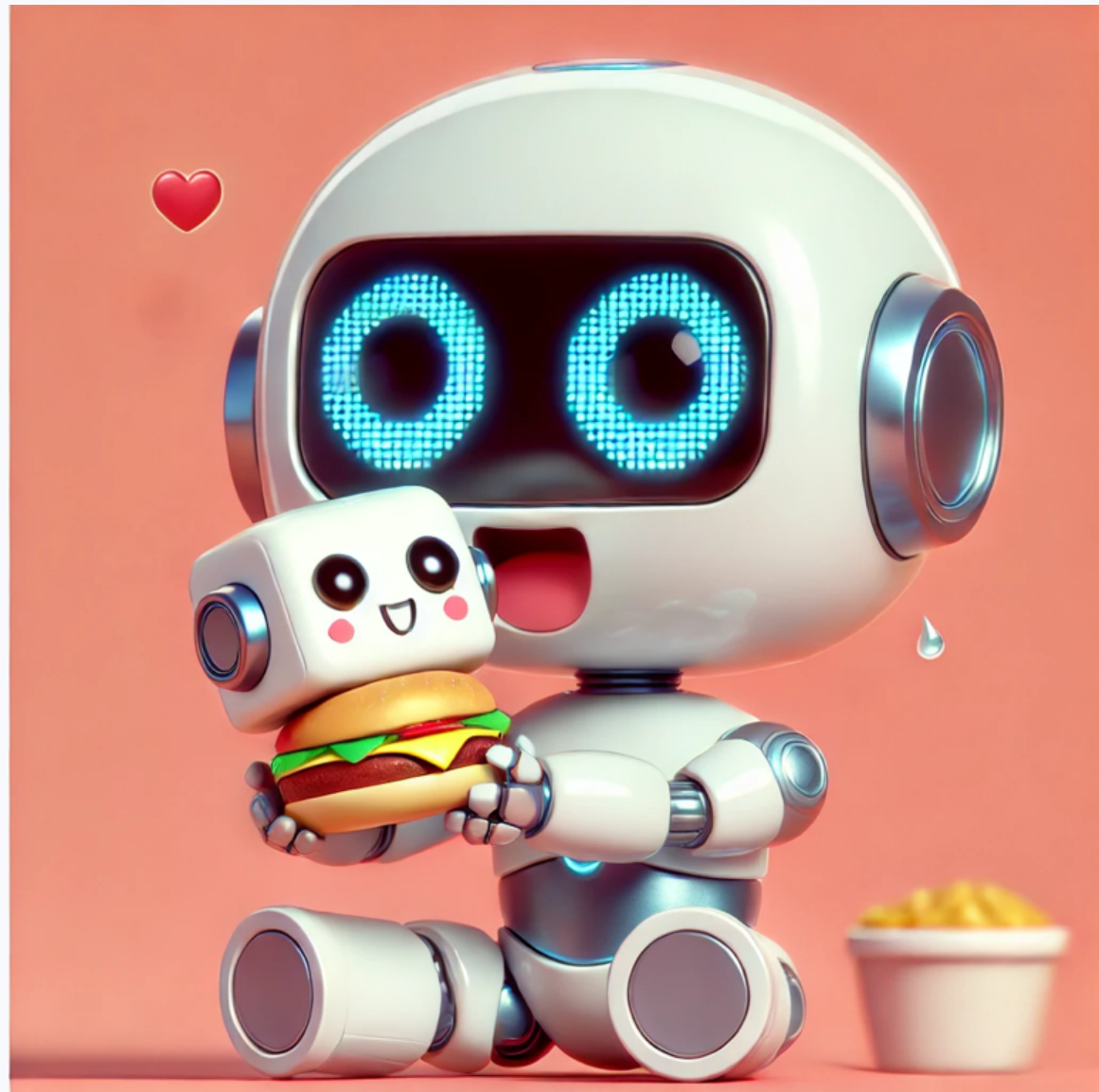
사과 ↔ 배 · 달리다 ↔ 뛰다 · 문법적으로 비슷한 단어



전이학습은 이미 배운 가중치에서 시작한다

- 처음부터 무작위 가중치로 시작하지 않는다
- 비슷한 문제를 학습한 모델의 가중치를 가져온다
- 더 적은 데이터와 시간으로 새 작업을 학습한다

출발선을 앞으로 옮기는 것과 같다.



사전 학습과 파인튜닝은 역할을 나눈다

단계	목표
사전 학습	대규모 데이터에서 일반적인 언어 패턴을 학습
전이	학습된 가중치를 새로운 작업의 출발점으로 사용
파인튜닝	작은 추가 데이터로 특정 목적에 맞게 미세 조정

경우에 따라 파인튜닝 없이도 사전 학습 모델을 활용한다

LLM은 여러 기능을 움직이는 공통 엔진이 된다

- LLM이 입력 문장을 풍부한 특성 벡터로 변환한다
- 작업별 모듈이 그 벡터를 받아 결과를 만든다
- 감성 분석, 분류, 개체명 인식, 질의응답으로 확장된다

하나의 엔진에 목적에 맞는 부품을 연결하는 구조



하나의 모델이 왜 여러 언어 작업에 **재사용**될까?

문장 표현 안에 의미, 문법, 관계, 맥락 정보가 함께 담겨 있기 때문이다.

4. 다음 단어 예측에서 트랜스포머로

문장 전체의 관계를 더 정확하고 빠르게 학습하다

언어 모델의 기본 문제는 “다음 단어는 무엇인가?”

나는 → 학교에 → 간다

- 문장에서 현재까지의 단어를 입력으로 사용한다
- 실제 다음 단어와 예측을 비교한다
- 더 잘 맞힐 때까지 가중치를 반복해서 조정한다
- 생성할 때는 예측한 단어를 다시 입력에 붙인다

단순한 목표를 대규모 텍스트에 반복 적용한다

대규모 언어 모델은 언어 체계를 담는 범용 모델로 확장됐다

- 훨씬 많은 텍스트를 학습한다
- 훨씬 많은 매개변수로 패턴을 표현한다
- 문장을 생성할 뿐 아니라 분석·분류·질의응답에도 활용된다
- 학습의 바닥에는 여전히 가중치와 편향이 있다

규모는 달라졌지만 학습의 핵심은 오차를 줄이는 방향으로 가중치를 바꾸는 것이다.

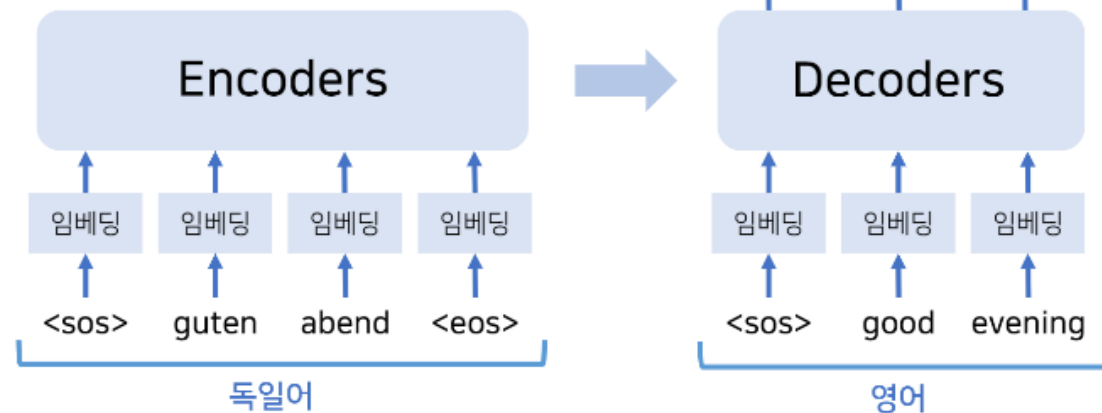
인코더는 읽고, 디코더는 생성한다

- **인코더**: 입력 문장을 숫자 표현으로 변환한다
- **디코더**: 그 표현을 바탕으로 출력 문장을 생성한다
- 번역처럼 입력 열과 출력 열이 대응되는 작업에 적합하다

나는 학교에 간다 → 인코더 → 표현 → 디코더 →

I go to school

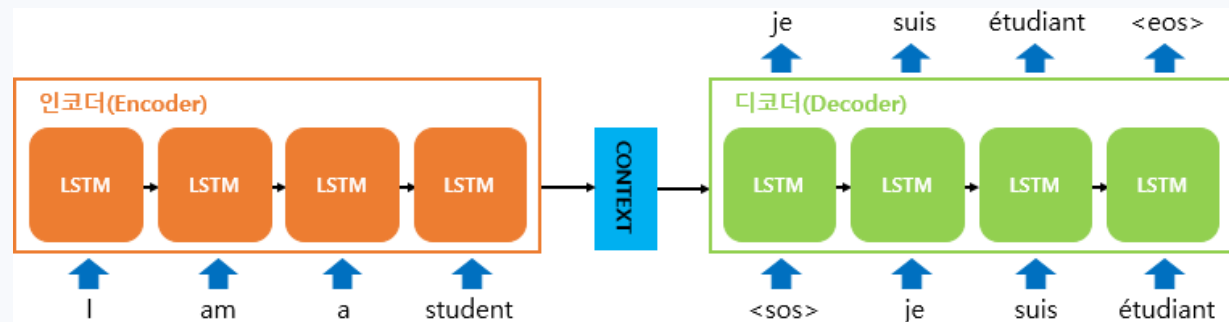
<eos>가 나올 때까지 디코더를 이용합니다.



컨텍스트 벡터는 문장 전체를 하나의 표현에 압축한다

- 인코더 RNN이 단어를 순서대로 처리한다
- 마지막 상태값에 문장 정보가 누적된다
- 디코더는 이 값을 초기 재료로 문장을 생성한다

단어 임베딩을 넘어 문장 전체를 임베딩한다



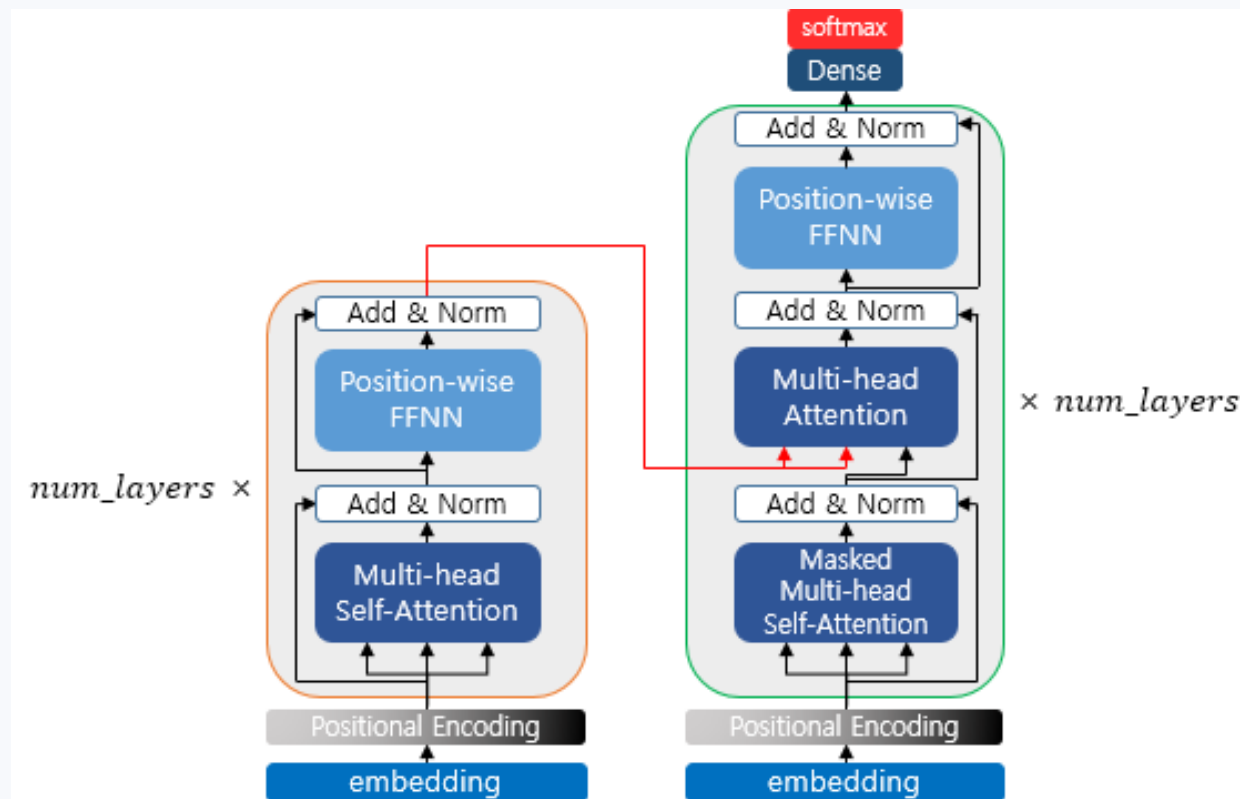
번역할 때 원문을 한 번만 보고 기억만으로 끝까지 번역
할까?

긴 문장일수록 필요한 단어를 다시 확인해야 한다.

어텐션은 단어를 생성할 때 입력 전체를 다시 본다

1. 현재 생성할 단어와 입력의 모든 단어를 비교한다
2. 관련성이 큰 단어에 더 높은 중요도를 준다
3. 중요도를 반영한 문맥 정보를 만든다
4. 그 정보를 사용해 다음 단어를 예측한다

순서가 아니라 문맥상 중요도에 집중한다



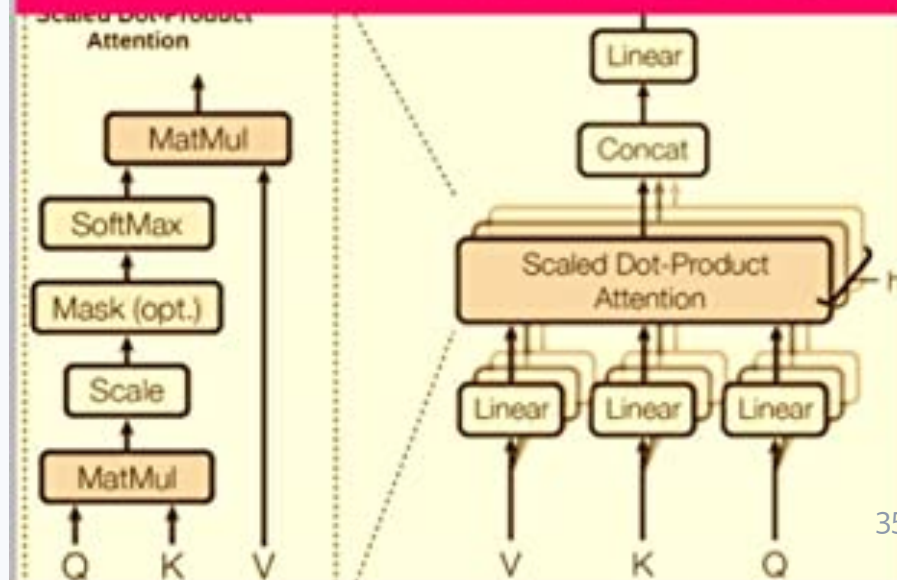
트랜스포머는 RNN 없이 어텐션을 중심으로 설계됐다

- 2017년 논문 *Attention Is All You Need*에서 제안
- 순환 상태 대신 각 토큰의 임베딩을 직접 사용
- 위치 정보를 별도로 더해 단어 순서를 표현
- BERT와 GPT 같은 대규모 언어 모델의 기반이 됨

Attention is All You Need

The Game-Changing Paper That Transformed NLP

HENRI VAN MAARSEVEEN



트랜스포머는 세 방향의 관계를 학습한다

어텐션	바라보는 범위	목적
인코더 셀프 어텐션	입력 문장 안의 모든 단어	입력 문맥 이해
디코더 셀프 어텐션	지금까지 생성한 단어	자연스러운 다음 단어 예측
인코더-디코더 어텐션	출력과 입력 사이	입력에 맞는 문장 생성

어텐션은 정확도뿐 아니라 학습 방식도 바꿨다

- RNN: 단어를 순서대로 처리해야 한다
- 트랜스포머: 여러 단어의 관계를 **병렬로** 계산할 수 있다
- 각 단어 표현이 문장 전체의 관계를 반영한다
- 인코더와 디코더를 여러 층으로 쌓아 표현력을 높인다

더 정교한 문맥 표현과 대규모 병렬 학습이 함께 가능해졌다.

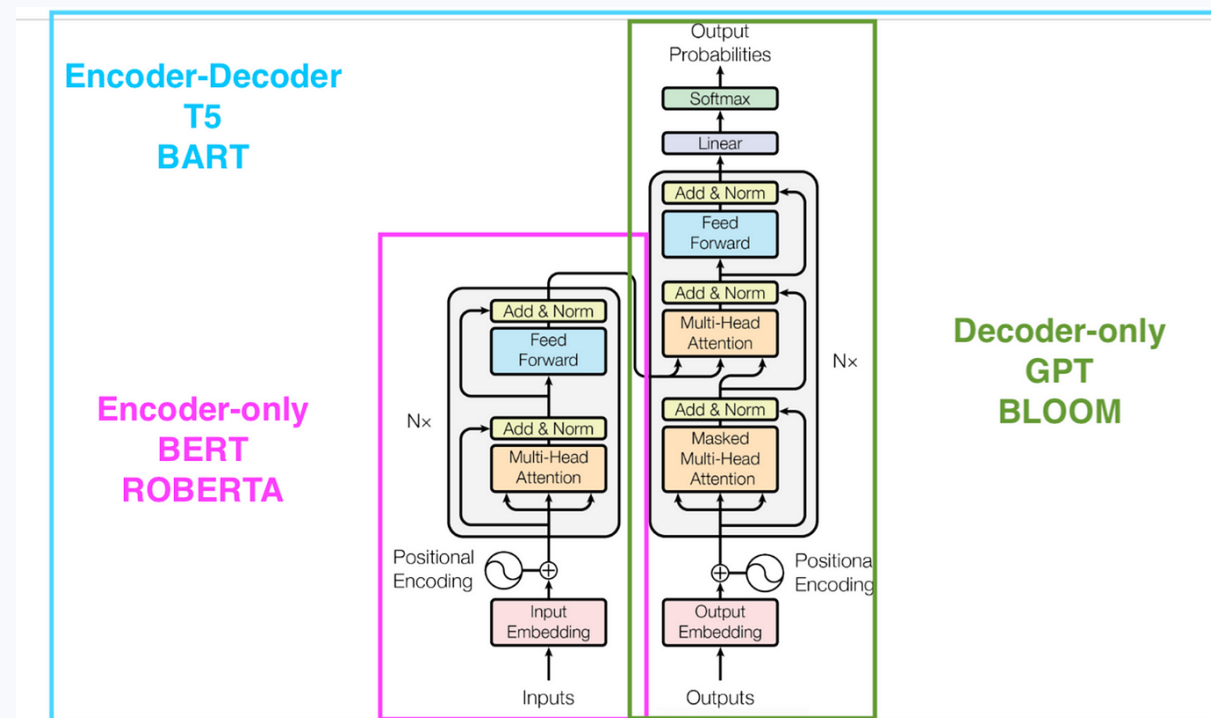
5. BERT와 GPT, 그리고 LLM의 현실

같은 트랜스포머에서 이해와 생성의 방향이 갈라지다

BERT는 이해에, GPT는 생성에 강점을 둔다

BERT	GPT
트랜스포머 인코더 중심	트랜스포머 디코더 중심
가려진 단어를 문맥으로 예측	이전 단어로 다음 단어를 예측
분류·검색·개체명 인식	문장 생성·대화·자동 완성

둘 다 대규모 사전 학습 후 다양한 작업에 전이된다



두 모델은 출력의 형태도 다르다

- **BERT 출력:** 각 토큰의 문맥화된 임베딩 벡터
- **GPT 출력:** 다음 토큰 후보들의 확률 분포

BERT → “이 단어는 이 문맥에서 어떤 의미인가?”

GPT → “이 문맥 다음에는 어떤 단어가 올 가능성이 큰가?”

고성능 LLM은 네 가지 조건이 함께 만든다

1. 트랜스포머 아키텍처

문장 전체의 관계를 정교하게 학습

2. 대규모 학습 데이터

다양한 언어 패턴과 지식을 경험

3. 많은 매개변수

복잡한 패턴을 표현할 수 있는 용량

4. 대규모 컴퓨팅 자원

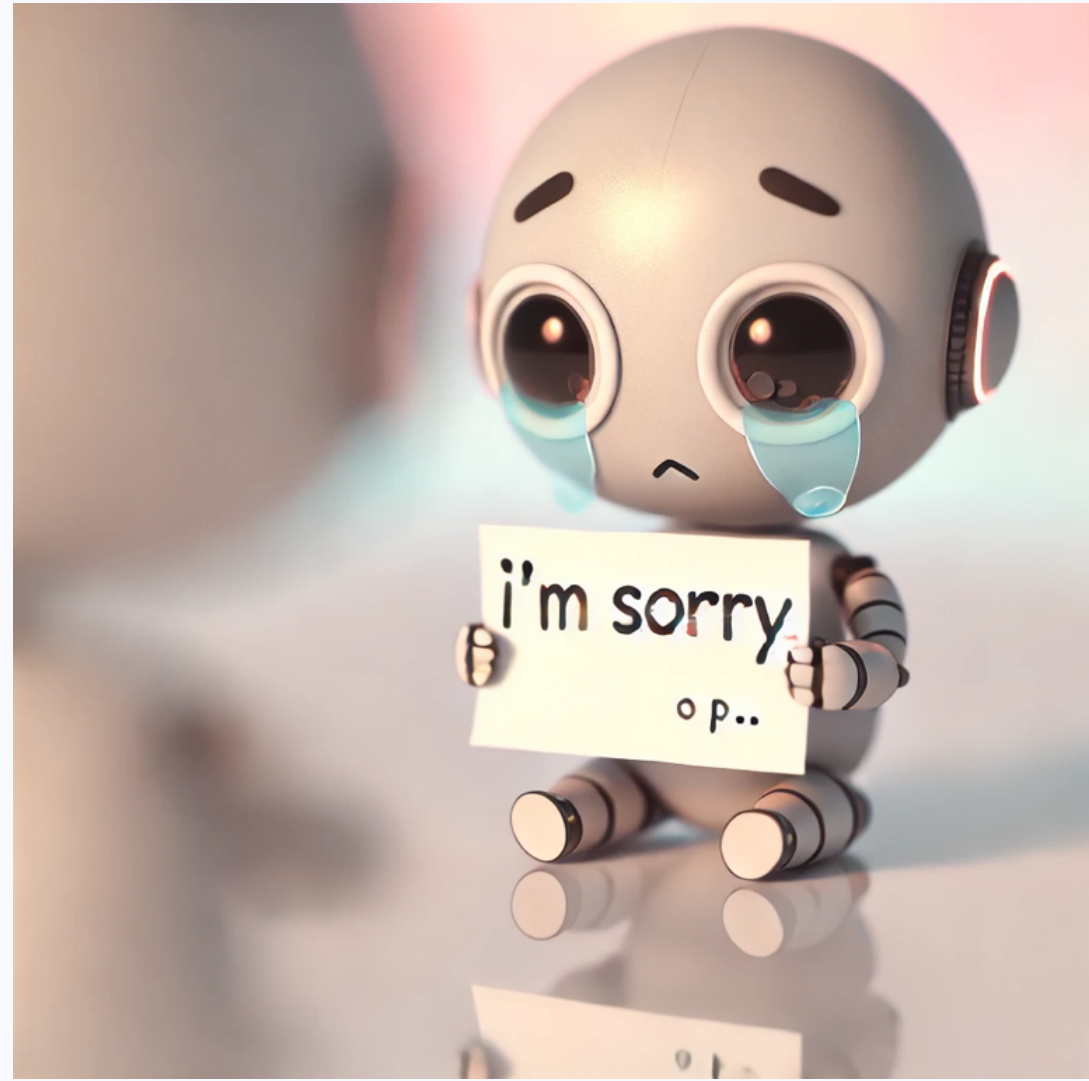
병렬 연산으로 거대한 학습을 현실화

구조만이 아니라 데이터·모델 용량·연산 자원이 함께 성능을 만든다.

LLM은 자연스럽게 말해도 사실을 보장하지 않는다

- 학습 목표는 **그럴듯한 다음 토큰 예측**이다
- 사실과 다른 내용을 자신 있게 생성할 수 있다
- 최신 정보나 특정 조직의 내부 지식이 부족할 수 있다
- 중요한 판단에는 출처 확인과 별도 검증이 필요하다

유창함과 정확성은 같은 개념이 아니다



RAG는 생성 전에 관련 근거를 검색한다

1. 사용자의 질문을 받는다
2. 신뢰할 수 있는 문서에서 관련 정보를 찾는다
3. 검색된 정보를 프롬프트의 근거로 제공한다
4. LLM이 그 근거를 바탕으로 답변을 생성한다

모델의 기억만 믿지 않고, 답변 시점에 외부 근거를 연결한다.

SLM은 더 작은 규모로 목적에 집중한다

- LLM보다 적은 연산과 메모리로 운영 가능
- 특정 언어·산업·업무에 맞춰 특화하기 좋다
- 비용, 지연 시간, 보안 요구에 대응할 수 있다
- 모든 범용 능력보다 **필요한 능력의 효율**을 우선한다

“가장 큰 모델”이 언제나 “가장 알맞은 모델”은 아니다

결국 다시 $H(x) = wx + b$

퍼셉트론도, DNN도, 트랜스포머도 가중치를 학습한다

복잡성은 커졌지만 핵심은 같다.

데이터를 보고 → 예측하고 → 오차를 줄이도록 가중치를 바꾼다.